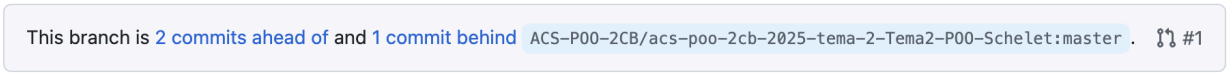
Tema 2 - POO 2025

Responsabil: Andreea Dutulescu
Concepte: Colecții, Genericitate [2], Excepții, I/O
Deadline: 14 ianuarie, ora 23:59.
Link GitHub Assignment: https://classroom.github.com/a/6Kxex2x2

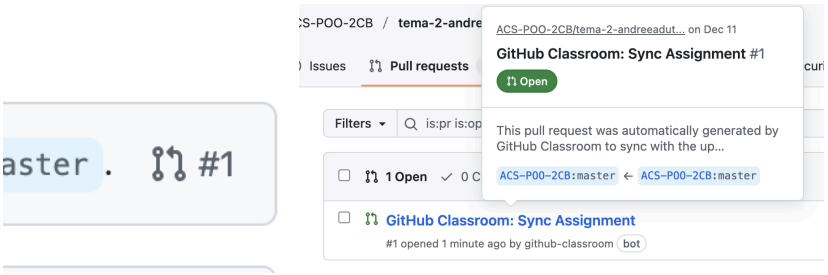
UPDATE 12.12.2025: Am rezolvat un typo în input-ul testului 12. Pentru actualizări, acceptați pull request-ul.

UPDATE 11.12.2025: Anumite teste aveau fișierele de referință greșite (console.ref și exam_report.ref). Am rezolvat problema. Pentru cei care nu ați acceptat invitația de classroom încă, nu trebuie să faceți nimic, când veți accepta invitația testele vor fi cele corecte. Pentru cei care deja ați acceptat invitația, ați primit un Pull Request pe care trebuie să-l acceptați pentru a vi se actualiza testele. Nu ar trebui să aveți merge conflicts, deci acțiunea ar trebui să fie finalizată cu succes. Mai jos sunt niște capturi de ecran cu ce aveți de făcut.

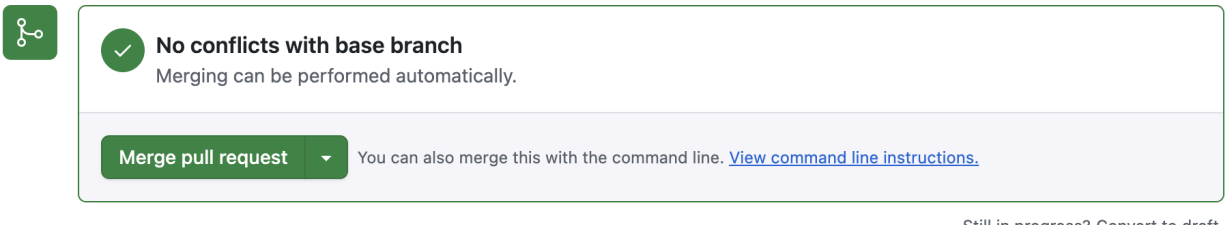
Vă va apărea că sunteți -1 commit behind-.



Dați click pe acel pull request.



Merge pull request.



Descriere generală

Sarcina voastră este să implementați o platformă simplificată pentru examinare online, asemănătoare cu Moodle, capabilă să gestioneze examene, întrebări, studenți și evaluări. Sistemul trebuie să poată interpreta comenzi primite dintr-un fișier text, în care fiecare linie conține o comandă emisă de frontend. Comenzile sunt în ordine cronologică în fișierul de input, în formatul `zz-ll-aaaa-hh-mm | <nume_comanda> | <parametru1> | <parametru2> | ... | <parametruN>`. Toate comenzile sunt delimitate prin `|` . Se garantează că acest caracter e folosit doar pentru delimitarea comenzilor și nu apare în interiorul lor.

Sistemul trebuie să mențină evidența:

- examenele create
- întrebările adăugate la fiecare examen
- răspunsurile studenților și scorurile obținute.

Aplicația va conține următoarele clase și interfețe principale (puteți adăuga clase/definiții noi sau schimba denumirea claselor, mai puțin clasei `Main`):

- `Main` - citește/parsează comenzile
- `Platform` - interpretează comenzile și gestionează obiectele principale
- `Gradable` (interfață) - definește comportamentul entităților care pot fi notate
- `Question` (clasă abstractă) - o clasă pentru toate tipurile de întrebări
- `OpenEndedQuestion`, `FillInTheBlankQuestion`, `MultipleChoiceQuestion` - extensii ale clasei `Question`
- `Exam` - reprezintă un examen
- `Student` - modelează un student și rezultatele sale
- `Correctness` (enum [1]) - o enumerare ce descrie gradul de corectitudine al unui răspuns
- `ResponseOption` (enum [1]) - o enumerare ce conține literele (opțiunile) unei întrebări grilă (A, B, C sau D)
- `SubmissionOutsideTimeIntervalException` (excepție) - o clasă de tip excepție care va conține informații despre timestamp-ul la care a apărut și o metodă `toString` pentru returnarea de detalii despre excepție.

Clasa Main

Această clasă citește fișierul cu comenzi, le interpretează și declanșează acțiunile corespunzătoare. Este de preferat să aveți cât mai puține funcționalități în această clasă (ex: ea doar să parseze și să citească comenzile din fișierul de intrare și să apeleze metodele platformei).

Clasa Platform

Ea gestionează funcționalitățile generale și coordonează comenzile primite de la clasa `Main` (crearea întrebărilor, examenelor, înscrierea studenților și generarea rapoartelor). Va conține atribute cu scop general (ex. colecții de examene și studenți, informații de printare, etc). În funcție de cum modelați aplicația, probabil metodele din această clasă vor corespunde ca funcționalitate cu tabelul din secțiunea *Comenzile*.

Interfața Gradable

Va conține o metodă abstractă care primește ca parametru corectitudinea unei rezolvări (eg. dacă a fost corectă, parțial corectă sau incorectă. Hint: Enum-ul `Correctness`). Metoda va returna un `double`, reprezentând procentul de punctaj pe care îl primește studentul pentru cele trei cazuri de corectitudine (ex. 1.0 pentru corect, 0.2 pentru parțial corect). În cadrul interfeței doar se definește antetul metodei, urmând ca funcționalitatea metodei să fie responsabilitatea claselor care implementează `Gradable`.

Clasa abstractă Question

Această clasă modelează funcționalitățile unei întrebări de examen în platformă. Ea implementează interfața `Gradable`, dar încă nu oferă o definiție concretă pentru funcționalitatea metodei din interfață.

Va avea următoarele atribute (private): textul întrebării (`String`), numele persoanei care a adăugat întrebarea (`String`), data la care sa adăugat întrebarea (`LocalDateTime` [3]), dificultate (`int`) și punctajul maxim asociat întrebării (`double`). Va conține metode de tip `getter` și `setter` pentru aceste atribute. De asemenea, va conține metoda abstractă `checkAnswer` care poate primi ca parametru un `String` sau un `ResponseOption`, urmând ca funcționalitatea concretă să fie dată în clasele care moștenesc `Question` (modelați posibilitatea de a primi fie un `String`, fie un `ResponseOption` prin genericitate. Exemplu general: `public abstract <T> returnType myFunction(T input);`). Funcția voastră va trebui să returneze dacă întrebarea a fost răspunsă corect, parțial corect sau greșit. Pentru întrebările grilă (`MultipleChoiceQuestion`), `T` este un `ResponseOption`. Pentru întrebările deschise (`OpenEndedQuestion` sau `FillInTheBlank`), `T` este un `String`.

Clasele OpenEndedQuestion, FillInTheBlankQuestion, MultipleChoiceQuestion

Aceste clase moștenesc clasa `Question`. Clasele vor avea ca atribute suplimentare un răspuns corect (care poate fi de tipul `ResponseOption` pentru `MultipleChoiceQuestion` și de tipul `String` pentru restul).
În plus, clasele acestea oferă o definiție/funcționalitate concretă pentru metoda abstractă de verificare a corectitudinii unui răspuns (din clasa părinte) și a metodei de returnare a procentului de punctaj (din interfață).
Metoda de verificare a corectitudinii unui răspuns este diferită ca funcționalitate în funcție de fiecare tip de întrebare.

- Pentru întrebările open-ended, un răspuns dat de student (de tip `String`) este:
    - corect dacă este egal răspunsul corect al întrebării
    - parțial corect dacă are numărul de litere în intervalul numărului de litere al răspunsului corect +-30% (cast la int), inclusiv și este un substring al răspunsului corect sau răspunsul corect e un substring al său.
    - greșit, altfel
- Pentru întrebările de tip fill-in-the-blank, un răspuns dat de student (de tip `String`) este:
    - corect dacă este egal răspunsul corect al întrebării
    - parțial corect dacă are numărul de litere în intervalul numărului de litere al răspunsului corect +-2, inclusiv și este un substring al răspunsului corect sau răspunsul corect e un substring al său.
    - greșit, altfel
- Pentru întrebările cu răspuns multiplu, un răspuns dat de student (de tip `ResponseOption`) este:
    - corect, dacă este aceeași literă ca răspunsul corect
    - greșit, altfel

De asemenea, aceste clase oferă o implementare pentru metoda din interfața Gradable, și anume, în funcție de corectitudinea primită ca parametru, vor returna un procent (între 0 și 1). Toate returnează 0 pentru incorect și 1 pentru corect. Pentru parțial corect, întrebările open-ended vor avea 70% (0.7), iar întrebările fill-in-the-blank vor avea 50% (0.5).

Clasa Exam

Această clasă modelează un examen, care conține mai multe întrebări asociate. Drept caracteristici, va avea o denumire (de tip `String`), o dată de început (de tip `LocalDateTime` [3]) și o dată de final (tot de tip `LocalDateTime` [3]), reprezentând perioada de desfășurare a

examenului. Este recomandat ca aceste atribute să fie private și accesate prin metode de tip `getter` și `setter`, pentru a respecta principiul încapsulării.
Clasa va include funcționalități pentru adăugarea unei întrebări în cadrul examenului. De asemenea, va oferi posibilitatea afișării întrebărilor la un moment dat, sortate cronologic în funcție de data la care au fost adăugate și, în caz de egalitate, alfabetic după numele persoanei care a creat întrebarea. Această funcționalitate este utilă pentru verificarea istoricului și pentru urmărirea evoluției conținutului examenului în stadiul de brainstorming.
Clasa va avea și o funcționalitate de tip „printare examen” pentru vizualizare sau generare de raport, în care întrebările sunt sortate inițial după nivelul de dificultate (crescător), iar pentru întrebările cu același nivel de dificultate, sortarea se va face alfabetic după textul întrebării. Această metodă facilitează afișarea clară și ordonată a întrebărilor atunci când examenul este pregătit pentru studenți.
De asemenea, clasa va menține o colecție de studenți care au participat la examen, împreună cu scorurile obținute de fiecare. Clasa va avea o funcționalitate pentru generarea unui fișier text (la orice moment de timp) care conține lista curentă a studenților și notele lor, ordonată alfabetic, la un anumit moment de timp, pentru a facilita raportarea rezultatelor examenului dar și verificarea în timpul examenului a cum se descurcă studenții.

Clasa Student

Această clasă modelează funcționalitatea unui student. Va avea ca atribute numele (`String`), grupă (`String`) și o colecție care să gestioneze examenele pe care le-a dat și scorul aferent.

Comenzile

Comenzile vor fi citite și parsate în cadrul clasei `Main`. Funcția `public static void main(String[] args)` va primi ca parametru numele aferent testului ce urmează să fie executat. Astfel, folderul aferent testului va avea denumirea parametrului, iar fișierul care conține comenzile se va afla în folderul aferent și va avea denumirea `input.in`. De exemplu, pentru parametrul `00-test`, folderul aferent va avea calea `"src/main/resources/00-test"`, iar fișierul cu comenzi va avea calea `"src/main/resources/00-test/input.in"`.
!!! Se garantează faptul că **nu** vor exista situații eronate în cadrul comenzilor, în afara situației de excepție menționată la comanda `submit_exam` (adică nu vor exista situații în care timestamp-urile să nu fie cronologic, în care să existe confuzie de nume la studenți sau examene, în care să nu existe un anumit student sau examen, etc).

Va trebui să creați și să scrieți rezultatul comenzilor ce necesită afișare în fișierele `"src/main/resources/00-test/questions_history_<nume_examen>_<timestamp>.out"`, `"src/main/resources/00-test/print_exam_<nume_examen>_<timestamp>.out"`, `"src/main/resources/00-test/exam_report_<nume_examen>_<timestamp>.out"` sau `"src/main/resources/00-test/console.out"`, iar fișierele de referință, pe care nu aveți voie să le modificați, vor fi de tipul `"src/main/resources/00-test/***.ref"`.

Comanda	Funcționalitate
`<timestamp> | create_exam | <nume_examen> |`	Se instanțiază un obiect corespunzător unui examen și se adaugă în colecția de gestiune a examenelor.

`<data_start> | <data_end>` Ex. `01-01-2025-10-00 | create_exam | POO_Final | 15-01-2025-09-00 | 15-01-2025-11-00`	
`<timestamp> | add_question | <tip_intrebare> | <nume_examen> | <autor> | <dificultate> | <punctaj_maxim> | <text_intrebare> | <raspuns_corect>` Ex. `01-01-2025-10-05 | add_question | multiple_choice | POO_Final | MihaiDascalu | 3 | 10 | Cât face 1+1? A)1 B)2 C)3 D)4 | B`	Se instanțiază și adaugă o întrebare de tipul specificat (`open_ended`, `fill_in_the_blank`, `multiple_choice`) la examenul dat. Setează autorul, dificultatea, punctajul maxim și răspunsul corect.
`<timestamp> | list_questions_history | <nume_examen>` Ex. `01-01-2025-10-10 | list_questions_history | POO_Final`	Listează întrebările adăugate la examenul indicat, sortate cronologic după dată și, în caz de egalitate, alfabetic după autor. Rezultatul se scrie în fișierul `questions_history_<nume_examen>_<timestamp>.out`. Consultați fișierele de test pentru a vedea formatul afișării (câte o întrebare pe fiecare rând). Pentru fiecare întrebare se va afișa, în această ordine, data de adăugare a întrebării, textul, răspunsul corect, dificultatea, punctajul maxim și autorul.
`<timestamp> | print_exam | <nume_examen>` Ex. `01-01-2025-10-15 | print_exam | POO_Final`	Afișează întrebările examenului în ordinea de dificultate crescătoare, iar în caz de egalitate, alfabetic după text. Se salvează în fișierul `print_exam_<nume_examen>_<timestamp>.out`. Consultați fișierele de test pentru a vedea formatul afișării (câte o întrebare pe fiecare rând). Pentru fiecare întrebare se va afișa, în această ordine, punctajul maxim, textul întrebării, dificultatea și răspunsul corect.
`<timestamp> | register_student | <nume_student> | <grupa>` Ex. `01-01-2025-10-15 | register_student | Ana Popescu | 321CB`	Creează un obiect `Student` și îl adaugă în sistem. Studenții sunt unici după nume.
`<timestamp> | submit_exam | <nume_examen> | <nume_student> | răspuns1 | răspuns2 | … | răspunsN` Ex. `01-01-2025-10-15 | submit_exam | POO_Final | Ana Popescu | A |`	Înregistrează răspunsul studentului pentru examenul menționat. Răspunsurile studentului la întrebări sunt în ordinea întrebărilor printate (sortarea corespunzătoare `print_exam`). Sistemul (în cadrul clasei `Exam`, într-o metodă specifică) verifică corectitudinea fiecărui răspuns (prin `checkAnswer`), calculează procentul de punctaj (prin `Gradable`) și punctajul final per fiecare întrebare și actualizează

`polimorfism | clase abstracte sau interfete | D`	scorul total al studentului pentru acel examen. În cazul în care timestamp-ul la care studentul dă examenul este în afara orelor definite ale examenului, se va arunca (de către metoda aferentă din `Exam`) o excepție de tipul `SubmissionOutsideTimeIntervalException`, scorul va fi automat 0 și se va afișa mesajul de excepție `<timestamp> | Submission outside of time interval for student <name>` în cadrul fișierului `console.out`.
`<timestamp> | show_student_score | <nume_student> | <nume_examen>` Ex. `01/01/2025 - 10:15 | show_student_score | Ana Popescu | POO_Final`	Afișează pe o linie nouă în fișierul `console.out`, scorul curent al studentului în examenul specificat, în formatul `<timestamp> | The score of <nume_student> for exam <nume_exam> is <score>.` Folosiți `String.format("%.2f", nr)` pentru a printa doar primele 2 zecimale.
`<timestamp> | generate_report | <nume_examen>` Ex. `01/01/2025 - 10:15 | generate_report | POO_Final`	Creează fișierul `exam_report_<nume_examen>_<timestamp>.out` care conține lista studenților și notele lor, ordonată alfabetic după numele studenților. Consultați fișierele de test pentru a vedea formatul afișării (câte un student pe fiecare rând). Folosiți `String.format("%.2f", nr)` pentru a printa doar primele 2 zecimale.
`<timestamp> - exit` Ex. `01/01/2025 - 10:15 - exit`	Nu mai apar comenzi după aceasta, se încheie programul.

Punctaj

*Se cere folosirea exclusivă a `Java Collections`. Nu se admit tablouri standard.
**Pe lângă atributele/metodele/clasele menționate în enunț, puteți adăuga voi altele care să vă ajute. Important este să existe și lucrurile cerute concret în enunț.
***Pentru modelarea și menținerea timestamp-urilor, este obligatoriu folosirea clasei `LocalDateTime` [3], existente deja în Java.
****Pentru o bună funcționare a aplicației, închideți fișierele de scriere (`.close()`) după ce s-a terminat de scris în ele/la finalul programului.

50 de puncte - Suita de teste
20 de puncte - Folosirea colecțiilor eficiente din punct de vedere al complexității temporale de execuție, potrivite pentru operațiile efectuate (găsirea unui examen/student după nume, diferite ordonări din cadrul examenului, etc).
Atenție! Operațiile cerute pot fi intercalate și pot apărea de mai multe ori, lucru care ar trebui să influențeze alegerea colecțiilor potrivite din punct de vedere al timpului de execuție (nu ne interesează memoria alocată). De exemplu, se poate da comanda `add_question` de mai multe ori, urmate de o comandă de `print_exam/list_questions_history`, urmate de alte adaugări de întrebări și apoi afișări. De asemenea, afișarea listei cu notele studenților poate

apărea de mai multe ori în timpul examenului (uneori cu noi studenți de afișat). Nu vor exista situații excepționale (ex: să se adauge o întrebare după timpul de începere al examenului).
5 puncte - Folosirea corectă și completă a genericității [2] în cadrul metodei abstracte (și implementărilor) de verificare a corectitudinii unui răspuns.
10 puncte - Definirea, aruncarea și tratarea corectă a excepției menționate.
10 puncte - Folosirea conceptelor de POO studiate (encapsulare, abstractizare, moștenire și polimorfism)
5 puncte - Readme detaliat în care să se specifice motivul alegerii unui anumit tip de colecție pentru un anume task
10 puncte - Bonus: Descrieți într-un fișier separat (`Bonus.md`) o nouă funcționalitate pe care o puteți adăuga aplicației. Pentru punctajul maxim al bonusului, funcționalitatea trebuie să presupună minim o clasă adițională sau 2 metode adăugate la o clasă existentă.

[1] https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html
[2] https://docs.oracle.com/javase/tutorial/java/generics/types.html
[3] https://docs.oracle.com/javase/8/docs/api/java/time/LocalDateTime.html . Pentru parsare mai ușoară, puteți folosi de exemplu metoda `parse(CharSequence text, DateTimeFormatter formatter)`.